

JAVA PROGRAMMING · DAY 4

Loop Iterations Deep Dive

char Loops, Conditional Sums, Multiplication Tables, Vowels, Nested Loop Patterns

8 Programs · Every Loop Traced Iteration-by-Iteration

1 Loop Concept with char Data Type

Characters in Java are secretly numbers underneath (their **ASCII/Unicode value**). This means you can use **+** and **-** on a **char** loop variable just like you would with **int**, and Java will jump between letters for you.

Program 1 — Print Z U P K F A (Step Backwards by 5)

Expected Output: Z U P K F A

```
public class Main {
    public static void main(String[] args) {
        for (char c = 'Z'; c >= 'A'; c -= 5) {
            System.out.print(c + " ");
        }
    }
}
```

CONSOLE OUTPUT

```
Z U P K F A
```

Iteration Trace

Pass	Value of c	Condition (c >= 'A')	Printed	Update (c = c - 5)
1	'Z' (90)	90 >= 65 → true	Z	c becomes 85 → 'U'
2	'U' (85)	85 >= 65 → true	U	c becomes 80 → 'P'
3	'P' (80)	80 >= 65 → true	P	c becomes 75 → 'K'
4	'K' (75)	75 >= 65 → true	K	c becomes 70 → 'F'
5	'F' (70)	70 >= 65 → true	F	c becomes 65 → 'A'
6	'A' (65)	65 >= 65 → true	A	c becomes 60
7	60	60 >= 65 → false	loop stops	

What's happening here?

- Every letter has a hidden number: **'A' = 65**, **'B' = 66** ... up to **'Z' = 90**. This is called its ASCII value.
- **c -= 5** subtracts 5 from that hidden number each time, which "jumps" the letter backwards by 5 positions in the alphabet.
- The loop keeps going as long as the letter's number is still ≥ 65 (i.e. still at or after 'A').

2 Sum of Numbers Using a Loop

Loops are great for adding up a range of numbers. Combining a loop with an **if** condition lets you add only the numbers that match a rule (like "only even numbers").

Program 1 — Sum of Even Numbers from 1 to 10

Expected: 2 + 4 + 6 + 8 + 10 = 30

```
public class Main {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 10; i++) {
            if ((i % 2) == 0) {
                sum += i;
            }
        }
        System.out.println("The Sum of 10 digits only even is : " + sum);
    }
}
```

CONSOLE OUTPUT

```
The Sum of 10 digits only even is : 30
```

Fixed from your original code: Systm.out.println → System.out.println (typo — missing the "e" in System).

Iteration Trace

i	i % 2 == 0 ?	Action	sum after
1	1%2=1 → false	skipped	0
2	2%2=0 → true	sum += 2	2
3	false	skipped	2
4	true	sum += 4	6
5	false	skipped	6
6	true	sum += 6	12
7	false	skipped	12
8	true	sum += 8	20
9	false	skipped	20
10	true	sum += 10	30

What's happening here?

- **i % 2** gives the remainder when i is divided by 2. It's **0** only for even numbers.
- The **if** inside the loop acts like a filter — only numbers that pass the check get added to **sum**.
- **sum += i;** is shorthand for **sum = sum + i;**

Program 2 — Sum of Numbers from 55 to 70

```
public class Main {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 55; i <= 70; i++) {
            sum += i;
        }
        System.out.println("Sum of numbers from 55 to 70 is : " + sum);
    }
}
```

CONSOLE OUTPUT

```
Sum of numbers from 55 to 70 is : 1000
```

What's happening here?

- This time there's no **if** — every number from 55 to 70 (16 numbers total) gets added, one per pass.
- Quick check: average of 55 and 70 is 62.5, and there are 16 numbers, so 62.5 × 16 = 1000 — matches our loop's answer!

3 Multiplication Table Using a Loop

Instead of typing out every line of a times-table, a loop can generate it for any number.

Program 1 — Multiplication Table of 2

```
public class Main {
    public static void main(String[] args) {
        int number = 2;
        for (int i = 1; i <= 5; i++) {
            System.out.println(number + " * " + i + " = " + (number * i));
        }
    }
}
```

CONSOLE OUTPUT

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
```

What's happening here?

- **i** runs from 1 to 5, and each pass prints **number × i**.
- The brackets around **(number * i)** make sure the multiplication happens before it's joined into the text with **+**.
- Want a full 1-10 table instead? Just change the loop condition to **i <= 10**.

Program 2 — Multiplication Table of 15

```
public class Main {
    public static void main(String[] args) {
        int number = 15;
        for (int i = 1; i <= 10; i++) {
            System.out.println(number + " * " + i + " = " + (number * i));
        }
    }
}
```

CONSOLE OUTPUT

```
15 * 1 = 15
15 * 2 = 30
15 * 3 = 45
15 * 4 = 60
15 * 5 = 75
15 * 6 = 90
15 * 7 = 105
15 * 8 = 120
15 * 9 = 135
15 * 10 = 150
```

What's happening here?

- Exact same loop structure as Program 1 — only **number** changed from 2 to 15.
- This shows the real power of loops: the same few lines of code can generate **any** table just by changing one variable.

4 Print the Vowels Between a-z

We can loop through every letter from 'a' to 'z' (just like we looped through numbers) and use an **if** to print only the ones that are vowels.

Program 1 — Print Vowels (a, e, i, o, u)

```
public class Main {  
    public static void main(String[] args) {  
        for (char c = 'a'; c <= 'z'; c++) {  
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {  
                System.out.print(c + " ");  
            }  
        }  
    }  
}
```

CONSOLE OUTPUT

```
a e i o u
```

What's happening here?

- The loop visits **every** letter from 'a' to 'z', one at a time — that's 26 passes in total.
- The **if** with || ("OR") checks the current letter against all 5 vowels; only a match gets printed.
- Non-vowel letters (like 'b', 'c', 'd') still get checked by the loop, they simply fail the **if** and print nothing.

5 Pattern Printing Using Nested Loops

A **nested loop** is a loop inside another loop. The **outer loop** controls how many rows are printed, and the **inner loop** controls what happens within a single row. This makes nested loops perfect for building shapes and patterns.

Program 1 — Right-Angled Star Triangle

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

CONSOLE OUTPUT

```
*
**
***
****
*****
```

Iteration Trace — Outer i controls the row, Inner j controls the stars in that row

Outer i	Inner j values (0 to i)	Stars Printed	Row Output
0	j = 0	1 star	*
1	j = 0, 1	2 stars	**
2	j = 0, 1, 2	3 stars	***
3	j = 0, 1, 2, 3	4 stars	****
4	j = 0, 1, 2, 3, 4	5 stars	*****

What's happening here?

- For **every single pass** of the outer loop (i), the **entire inner loop** (j) runs from start to finish before moving to the next i.
- The inner loop's condition **j <= i** is the trick: as i grows bigger (row 0, 1, 2...), the inner loop gets to run more times, printing more stars.
- System.out.println();** (empty) is placed **outside** the inner loop but **inside** the outer loop — it moves to a new line only after a full row of stars is printed.

Program 2 — Alphabet Pyramid Pattern

Expected Output: A / AB / ABC / ABCD / ABCDE / ABCDEF / ABCDEFG

```
public class Main {
    public static void main(String[] args) {
        for (int i = 0; i < 7; i++) {
            for (int j = 0; j <= i; j++) {
                char letter = (char) ('A' + j);
                System.out.print(letter);
            }
            System.out.println();
        }
    }
}
```

CONSOLE OUTPUT

```
A
AB
ABC
ABCD
ABCDE
ABCDEF
ABCDEFG
```

Iteration Trace

Outer i	Inner j values	Letters ('A' + j)	Row Output
0	0	A	A
1	0, 1	A, B	AB
2	0, 1, 2	A, B, C	ABC
3	0, 1, 2, 3	A, B, C, D	ABCD
4	0-4	A-E	ABCDE

5	0-5	A-F	ABCDEF
6	0-6	A-G	ABCDEFG

What's happening here?

- This reuses the **exact same nested-loop skeleton** as Program 1 — only the inner line changed from printing "*" to printing a letter.
- **(char) ('A' + j)** takes the ASCII number of 'A' (65), adds **j** to it, then converts the result back into a character. When j=0 we get 'A', j=1 gives 'B', and so on.
- This is the same ASCII trick from Topic 1, now used **inside** a nested loop to build a growing sequence of letters.